SN Computer Science (2024) 5:821
https://doi.org/10.1007/s42979-024-03134-3

ORIGINAL RESEARCH

On the Impact of Input Models on the Fault Detection Capabilities of Combinatorial Testing

Carmen Baumann[1] · Yavuz Koroglu[1] · Franz Wotawa[1]

Received: 21 November 2023 / Accepted: 12 July 2024
© The Author(s) 2024

Abstract

Testing is an important activity to detect faults before software deployment. We focus on black-box combinatorial testing, where fault detection is one of the main objectives. In this paper, we argue that input model abstraction notably impacts the fault detection capability of a combinatorial test suite. First, we present experiments from previous work that support this argument. We then perform new experiments on a more diverse set of programs. These experiments use mutation testing to estimate fault detection capability, but we also include structural coverage measures in the new experiments. Finally, we elaborate on two possible improvements to obtain an optimal input abstraction strategy for not just continuous but all input domains. Both experiments suggest that input abstraction affects the fault detection capability. We claim that the improvements will produce a better input abstraction with which we can achieve better fault detection capability without increasing the test suite size.

Keywords Combinatorial testing · System testing · Input modeling impact

Introduction

Testing is, without any doubt, an important activity of software and system verification and validation for fault detection before deployment. Many methods and techniques exist for automating testing from test case generation to execution. In this paper, we focus on automated test case generation for software and systems assuming a black-box approach, i.e., no knowledge about the internal implementation. For black-box testing, only the interface and, to some extent, requirements and specifications are available. Black-box testing methodologies that gain a lot of attention include model-based testing (MBT) [1, 2], and combinatorial testing (CT) [3, 4]. Both approaches utilize models for test case generation. Whereas MBT requires models capturing the behavior of a system, CT solely relies on an input model comprising information about input and configuration parameters together with their domains.

MBT and CT apply to almost any system under test (SUT). Hence, it is understandable why MBT and CT saw use across multiple systems like mobile and telecommunication applications [5, 6], security [7–9], artificial intelligence and data mining [10, 11], and automotive (including autonomous driving) [12, 13]. Due to their widespread use, both approaches seem to provide test suites of sufficient fault detection capabilities for many practical applications. Hence, the question regarding factors influencing fault detection arises. Since both approaches rely on models, their quality seems to be an important factor. Indeed, a case study [14] reveals that input modeling was the most challenging undertaking in a practical CT application. Incorrect, incomplete, or redundant models would hamper the effectiveness of CT [15]. Moreover, in a recently published paper [16], the authors discuss the influence of input models on CT-generated test suites considering audio plug-ins as SUTs. This work shows in continuous input domains that some input

Carmen Baumann and Yavuz Koroglu have contributed equally to this work.

✉ Franz Wotawa
wotawa@ist.tugraz.at

Carmen Baumann
carmen.baumann@student.tugraz.at

Yavuz Koroglu
ykoerogl@ist.tugraz.at

[1] Christian-Doppler Laboratory for Quality Assurance Methodologies for Autonomous Cyber-Physical Systems, Institute of Software Technology, Graz University of Technology, Inffeldgasse 16b/2, 8010 Graz, Styria, Austria

Published online: 27 August 2024

abstractions yield a CT test suite with higher fault detection capability than others. Therefore, selecting the correct input abstraction strategy is essential to maximize the fault detection capability we can get from CT when the input domain is continuous.

CT cannot generate tests before the input model is available. Naturally, existing research on input modeling puts the modeling at the beginning of the testing process. So, the resulting input model depends on the general requirements of CT [17] and the apriori information on the SUT (e.g., ontologies [18, 19]). Also, input model generation strategies exist for general-purpose testing that are usable in CT [20, 21]. Again, these depend on the apriori information.

Note that having enough apriori information for coming up with relevant input models for CT cannot always be guaranteed. For example, when applying ontology modeling for advanced and autonomous driving functions (see [18, 19]), concrete values like distances between obstacles and the ego vehicle (i.e., the SUT) or the velocity of cars have to be abstracted. Depending on this abstraction, faults might be detectable or not. Despite the fact that CT provides superior fault detection capability compared to search-based testing (see [22]), abstraction and, afterward, concretization of values have not been considered. Hence, there is a need to investigate further the impact of input modeling on the fault detection capabilities of CT.

This paper summarizes previous work [16] on test suites for three audio plugins generated by the same CT approach but with different input domain models as a motivating example in Sect. "Background". Afterward, we provide a newer experimental evaluation in Sect."Experiments and Evaluation" considering different programs comprising input parameters like numbers and strings. In this evaluation, we are interested in the impact of domain size and selection on fault detection capabilities. We use mutation testing (MT) to estimate the fault detection capabilities of CT test suites. Then, we show a significant change in line coverage (LC) and mutation score (MS) when the input domain size is changed. These new results suggest that the inputs selected influences fault detection capability for both continuous and discrete domains. Hence, we review the research avenues in the previous work and describe possible ways to automate input abstraction towards higher fault detection capability. To the best of our knowledge, this is the first study that evaluates the influence of continuous and discrete input domain abstractions on CT fault detection capability, with the most extensive experimental program set for this purpose.

We structure this paper as follows. Section "Background" describes the motivating experiments in the previous work, while outlining the open questions from that work. We then present the new experiments that address these open questions and discuss the additional results in Sect. "Experiments and Evaluation". We continue discussing the automation and optimization of input abstractions for continuous and discrete input domains in Sect. "Discussion". Finally, we conclude with a summary of our findings and future work in Sect. "Conclusions".

Background

We now provide the necessary definitions for understanding the motivating experiments of the previous work through Sects. "Combinatorial Testing (CT)" and "Mutation Testing and Structural Coverage Criteria". Then, we discuss the experiments, results, and shortcomings in Sect. "Audio Plugin Experiments".

Combinatorial Testing (CT)

CT is a software testing approach that investigates software under various combinations of input values. In this context, SUT is a system that takes some inputs and produces some outputs. The space of all possible values of an SUT input is an input domain. An input model describes how many inputs an SUT has and defines the input domain for every SUT input. In some SUTs, the input model also describes impossible input combinations, further restricting the input domains under certain constraints. Note that an input model does not have to represent all possible input values of an SUT. Depending on the scope of testing, the input model may omit inputs or input values or include more constraints.

Given an input model for an SUT, the need to test it for different combinations of input values arises. A CT approach (e.g. IPOG-D [23] implementations such as ACTS [24] or our implementation) then generates a covering array for the SUT. A covering array lists input value combinations that satisfy a CT coverage criterion. A common CT coverage criterion is pairwise testing, which requires the covering array to include every input value pair. We say that for a pairwise coverage criterion, the combinatorial strength (k) of CT is two. We can strengthen this criterion by covering every triple ($k = 3$), quadruple ($k = 4$), or more, eventually including every possible combination.

CT cannot generate a test suite with a finite number of tests if one of the inputs has an infinite domain. Fortunately, we can assume a maximum bit width for every input type, making all input domains finite. Still, the relation between the input domain and test suite sizes is not linear [23]. Input abstraction is a common way to keep the test suite sizes manageable in CT. An abstracted input domain is a set of subintervals representing the input domain. So, CT generates fewer tests using an abstract input model. A practical way of input abstraction is input selection. Input selection creates abstracted input domains by selecting a finite number of representative input values from the input domain, allowing

CT to generate concrete and ready-to-execute tests [15]. Typically, known input model generation strategies such as category-based partitioning [20] suggest how can we perform input selection based on apriori information about the SUT. Literature consists several improvements to a typical combinatorial modeling, like lattice based semantics [25]. These improvements aim to facilitate manual input modeling, whereas we aim to automate input selection. Automating a lattice based combinatorial modeling design also remains an open question. The study gets closest to automating input modeling attempts to automatically update a manually created input model as the SUT is updated [26].

Mutation Testing and Structural Coverage Criteria

Without knowing the faults in an SUT, assessing its fault detection capability is still possible. We estimate the fault detection capability of a test suite via mutation testing (MT) [27]. The intention behind MT was to evaluate the quality of test suites. With carefully designed mutation operators, MT injects changes that mimic faults into the SUT. Then, the adequacy criterion for any test suite is to kill mutants, i.e., reveal the injected faults by executing the mutated parts of the SUT. Using MT the quality of a test suite can be evaluated by computing the fraction between the number of killed mutants and the total number of mutations. This fraction is called the mutation score. A high mutation score indicates that a test suite can detect more small deviations of a program and hence has better fault detection capabilities.

There are many tools for MT in the literature, like PIT [28], or MuJava [29]. These tools target specific programming languages, whereas *universalmutator* [30] works across programming languages. For more information on MT, we refer the interested reader to [31, 32] and more recently [33].

A structural coverage criterion ensures the testing of certain parts of the SUT's source code and is still an active research area [34]. The idea behind structural coverage is to find statements, branches, or paths that are executed on a given test suite. Hence, structural coverage like line or statement coverage, is, like the mutation score, an indicator of the test suite adequacy. However, low structural coverage indicates many parts of the SUT remain untested. Hence, all faults in those parts remain hidden. The common structural coverage criterion is line coverage (LC).

Audio Plugin Experiments

We now describe the experiments on three real-world audio plugins that reveal the influence of continuous domain input abstractions on the fault detection capability of CT test suites [16]. These audio plugins are *Freq.*, *Comp*, and *EQ2*. Of these three plugins, *Freq.*[1] is open-source and available online, whereas the others are commercial.

The three plugins under test receive an audio file and many settings parameters as input. For each plugin, the settings parameters comprise many floating-point numbers between 0.0 and 1.0, as well as some Boolean and integer variables. The experiments focus on the floating-point variables because they represent a continuous input domain that requires input abstraction. The Boolean and integer settings parameters are not abstracted because they have finite input spaces, and the input parameter model limits them to only a few choices. The audio file is constant.

The floating-point variables must be abstracted because CT cannot generate tests for infinite input domains. So, the purpose of abstraction here is to limit the input space to a finite number of representative input values. Hence, we use input selection. The experiments comprise two selection types, UniSamp and ApriSamp. UniSamp selects representatives uniformly, i.e., with equidistance between every consecutive input value, whereas ApriSamp uses values apriori known to the developers/testers. Hence, we expect ApriSamp to yield higher fault detection capability.

The number of representatives in the input selection can also influence the fault detection capability. These numbers in the experiments are five and eleven for Sparse and Fine sampling, respectively. So, for example, a Sparse UniSamp selection divides the continuous interval $[0.0, 1.0]$ into $\{0.0, 0.2, 0.4, 0.6, 0.8, 1.0\}$. A Fine UniSamp selection divides the same interval into $\{0.0, 0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 1.0\}$.

Table 1 shows the experimental results for the three audio plugins. The advanced combinatorial testing system (ACTS) [24] tool generated the test suites (T) for these plugins in Table 1. Then, the *universalmutator* [30] tool mutates the plugins under test and measures the mutation score (MS) by executing the test suites on the mutated plugins to estimate fault detection capability.

The results show that the ApriSamp abstraction yields equal or higher mutation scores except only when k is 2 for *Comp*. So, the results suggest an optimized input abstraction exists. However, these results may not apply to any arbitrary software under test, i.e., an optimal input abstraction strategy may not exist for software other than audio plugins. Moreover, the abstraction strategy was limited only to continuous domain inputs. In general, inputs could be integers or strings, too. So, the research avenues suggested in the previous work fail to consider domain input abstractions other than continuous ones. We do not know the structural coverage measures of the

[1] Frequalizer. url: https://github.com/vallant/reta. accessed: 16.11.2023.

Table 1 Results of the audio plugin experiments [16]

		Fine				Sparse			
	Audio	UniSamp		ApriSamp		UniSamp		ApriSamp	
k	Plugin	MS	|T|	MS	|T|	MS	|T|	MS	|T|
$k = 2$	*Freq.*	75.5	339	80.1	317	60.4	92	80.3	180
	Comp	45.6	336	57.0	312	48.4	92	40.4	90
	EQ2	46.9	400	47.7	358	42.4	109	47.6	230
$k = 3$	*Freq.*	76.3	7305	79.3	6592	61.2	1004	78.5	2792
	Comp	51.0	6,918	52.3	3803	50.1	995	50.2	830
	EQ2	–	–	–	–	48.2	1224	48.3	2340

k combinatorial strength, *MS* mutation score, |*T*| test suite size

Table 2 Description of the first five programs used for the experiments

Program	LOC	Class	Parameters	Description
BMI	23	1	weight:double, height:double	Calculates the body mass index based on two input parameters
Middle	27	1	a:integer, b:integer, c:integer	Takes three input parameters and calculates which of the three values is the mean value
CalDate	33	1	day:integer, month:integer, year:integer	Takes a date and calculates from it the Julian date
Remainder	45	1	a:integer, b:integer	Takes two input parameters and calculates the remainder
ColorHelper	62	1	col:Color, inc:integer	The first input parameter is a colored object. From this object, the red, blue, and green values are taken and incremented by the second parameter

resulting test suites. So, we do not know if large parts of the SUT are untested. Two audio plugins are unavailable online, making two-thirds of the experiments not reproducible. Finally, the experiments did not include a randomly selected input abstraction strategy, so the baseline fault detection capability remains unknown. Note that for *EQ2*, it was computationally impossible to run $k = 3$ tests because the SUT is slow and there were too many tests to execute in reasonable time. Nevertheless, the impact of input selection over mutation scores remains a fact.

Experiments and Evaluation

To evaluate the impact of the input model on the fault detection capabilities of CT, we carried out an experimental evaluation comprising 10 Java programs with 23–104 lines of code (LOC), considering different test suite generation scenarios and selection methods. In the following subsections, we describe the programs, the test scenarios, the experimental setup, and the obtained results.

Programs Used for Evaluation

Tables 2 and 3 summarize the 10 Java programs used for evaluation, including their parameters and purpose. Among the programs chosen, several were developed by Gordon Fraser [35], while the Airline and Clean programs were obtained from the Software Artifact Infrastructure Repository (SIR) for Experimentation [36]. The SIR repository is one of the most popular software testing repositories [37], where 60 percent of regression testing studies use this repository in their experiments [38].

We see from Tables 2 and 3 that each program comprises 2–3 input parameters of types integer, long, double, string, and Color. Although most of the underlying programs are small, they have inputs of data types, e.g., integers, where we can easily vary their domain, and considering all values for test case generation is not feasible.

For these example programs, we developed several test scenarios where we varied the domain of the inputs for testing. We use different sizes of domains and also consider different selection criteria for particular values. In

Table 3 Description of the last five programs used for the experiments

Program	LOC	Class	Parameters	Description
MulAndCheck	68	1	a:long, b:long	The product of two numbers is calculated, and it is checked if there is an overflow
Airline	83	2	numberThreads:integer, cushion:integer	Sells airline tickets to passengers. The first parameter indicates how many passengers want to buy a ticket simultaneously, and the second parameter indicates the number of seats on the plane. The program runs until all seats are sold
Luhn	91	1	number:String, type:integer	Validates the given credit card information
Clean	102	4	iFirstTask:integer, iSecondTask:integer, iterations:integer	Starts the given number of tasks for the given iterations
wrongLock	104	4	iNum1:integer, iNum2:integer	Takes two input parameters. The first one is the number that has a lock on the class, and the second one is the number of threads that have a false lock

Table 4 The intervals of the input variables of each program used for the experiments

Program	Parameters	Intervals
BMI	Weight	[0.5, 2.2]
	Height	[20.0, 120.0]
Middle	a	[−729432887, −767569861]
	b	[161338521, 1894441]
	c	[−968334698, −94888208]
CalDate	day	[1, 1000]
	Month	[1, 1000]
	Year	[1000, 2020]
Remainder	a	[36114, 59101]
	b	[53463, 69617]
ColorHelper	col	–
	inc	[1120873317, 1120874317]
MulAndCheck	a	[−7964412045037552000, −7964412045037551000]
	b	[7863516506798420000, 7863516506798421000]
Airline	numberThreads	[1, 1000]
	cushion:integer	[1, 10000]
Luhn	number	[29985954742903, 47973665697157587]
	type	[0, 1000]
Clean	iFirstTask	[1, 1000]
	iSecondTask	[1, 1000]
	iterations	[1, 1000]
wrongLock	iNum1	[1, 1000]
	iNum2	[1, 1000]

the following subsection, we outline the generation of the test scenarios in detail.

Test Scenarios

In the context of this study, a test scenario is a pair of domain size and input selection method. For example, a domain of 10 values selected randomly would be a test scenario. Several test scenarios were executed for each Java program to measure the influence of different domain sizes on the fault detection capabilities.

We applied each scenario to all input variable domains for every experiment except for string, and class instance type variables. For strings and class instances (e.g. color), we were bound by the size of the input space, which was smaller than the domain sizes in the scenarios. It is also worth noting that for numbers, including the credit card number in the Luhn program, we used a pre-selected interval for all experiments. Table 4 summarizes the intervals for each program

Table 5 Line coverage (*LC*) and mutation score (*MS*) obtained for the example programs using equal selection domains and a combinatorial strength of 2

	Scenario 1		Scenario 2		Scenario 3		Scenario 4	
Program	LC	MS	LC	MS	LC	MS	LC	MS
BMI	100	94	100	100	100	100	100	100
Middle	25	3	33	16	33	16	35	20
CalDate	94	84	94	84	94	88	100	100
Remainder	46	20	46	24	46	20	56	25
ColorHelper	65	40	100	100	100	100	100	100
MulAndCheck	35	19	40	19	45	23	50	27
Airline	100	8	100	8	100	25	–	–
Luhn	46	38	68	43	100	73	100	77
Clean	97	26	97	26	100	30	–	–
WrongLock	97	49	97	49	100	55	–	–

A '–' indicates that a program was not executable for some test cases

and input variable. For class instances and strings, there is no interval given.

In the following, we describe the scenarios and their selection process of the values, which can be either random or equal. Random selection utilizes a uniform distribution on the intervals, whereas equal selection obtains values starting with the beginning of an interval and keeping the distance between the following selected elements equal.

Scenarios 1–4

For scenarios 1–4, we introduced a domain size of 25, 50, 100, and 1000, respectively. These domain sizes are for random and equal selection. We now introduce three input types; numbers, strings, and class instances.

Numbers: For numbers, i.e., integers, floats, or doubles, *random selection* randomly takes n elements from a uniform distribution over the interval. In case of *equal selection*, we take n elements $e_1, \ldots, e_n$ from the interval $[a, b]$ as follows: $e_1 = a, \forall_{i=2,\ldots,n-1} : e_i = e_{i-1} + \delta, e_n = b$ where $\delta = \frac{b-a-1}{n-2}$. In the case of integers, we assume δ to be rounded to the next largest integer value.

Strings: Only the Luhn program comprises a string input. However, this input is a credit card number. Therefore, we handled it like a number in both scenarios.

Class instances: We do not distinguish random and equal selection for class instances. We analyzed the program and obtained several valid instances. For the different scenarios and the only program ColorHelper with a class instance parameter *col* we take 6, 8, 10, and 13 instances for scenarios 1–4, respectively.

Note that for all equal selection scenarios, we considered values outside the definite space, i.e., where a program must return an error message or use reasonable other mitigation actions. This was done for CalDate and parameter *year*, and MulAndCheck and the variables a and b.

Measurements

For obtaining line coverage and mutation score, where the latter is used to indicate the fault detection capabilities of the test suite, we rely on the PIT[2] designed for Java programs. To use PIT, developers can easily download it as a plugin and run it from the command line. The tool supports different mutators responsible for making small changes (mutations) to the original code, simulating potential defects or errors. The mutators automatically apply these mutations to the program, creating a set of faulty code versions. Note that all measurements in the experimental evaluation have been carried out using PIT. The outcomes of the mutation testing tool comprise line coverage and mutation score, explained in Sects. Line Coverage and Mutation Score, respectively.

Line Coverage

The line coverage (LC) metric indicates the percentage of lines of code in the program that are covered by the existing test cases. A high LC suggests that a significant portion of the code is exercised during testing, while a low value of LC may indicate areas of the code that lack sufficient testing.

Mutation Score

The mutation score (MS) denotes the count of mutants that are eliminated by the test cases. A mutant is considered *killed* in mutation testing if the corresponding test cases detect the introduced fault. MS is the ratio of killed mutants to the total number of mutants generated during testing. A higher MS suggests a more effective test suite, as it indicates that the tests can successfully detect a greater proportion of introduced faults.

[2] MT Tool PIT. 2023. url: https://pitest.org/. accessed: 30.07.2023.

Table 6 Line coverage (*LC*) and mutation score (*MS*) obtained for the example programs using random selection and a combinatorial strength of 2

	Scenario 1		Scenario 2		Scenario 3		Scenario 4	
Program	LC	MS	LC	MS	LC	MS	LC	MS
BMI	100.0	77.6	100.0	77.4	100.0	80.7	100.0	99.5
Middle	57.4	46.4	57.4	46.6	57.8	47.2	57.6	47.4
CalDate	83.2	75.9	83.2	75.9	93.6	82.8	78.0	71.5
Remainder	45.1	18.8	45.1	19.1	58.3	33.7	64.0	38.4
ColorHelper	64.3	38.8	64.3	39.0	99.6	99.6	99.9	99.9 5
MulAndCheck	64.5	45.2	64.5	45.4	43.0	25.7	43.0	16.4
Airline	100.0	15.6	100.0	15.5	–	–	–	–
Luhn	49.8	45.0	49.8	45.2	45.4	40.2	41.2	35.2
Clean	99.5	21.1	99.5	21.2	–	–	–	–
WrongLock	95.5	48.4	95.5	48.5	–	–	–	–

A '–' indicates that a program was not executable for some test cases

Evaluation Process

The evaluation process comprises computing all scenarios as described in Sect. "Test Scenarios" for all programs listed in Tables 2 and 3. Afterward, a combinatorial test suite of strength two is computed for each scenario and program. We used our Java implementation of the IPOG algorithm [39] for computing the test suites in Tables 5 and 6. All test suites are executed on their corresponding programs, and the measurements, i.e., the line coverage and mutation score, from Sect. "Measurements" are recorded. For the equal selection this process is carried out only once, since it is deterministic. For the random selection we carry out the process ten times and compute the average values. Hence, the overall evaluation process for obtaining experimental results can be summarized as follows:

1. For each program Π described in Tables 2 and 3 do:
   (a) For each scenario s and element selection process, i.e., random and equal, do:

(i) Compute all domains for the input variables of program Π using the specification of scenario s under the element selection process (random or equal). The input variables and their corresponding domains form the input model M.
(ii) Generate a test suite T of strength 2 for M using combinatorial testing. Note that T consists abstract values at this point.
(iii) Generate n concretizations of T where n is 1 and 10 for equal and random selections, respectively.
(iv) Execute all tests from T on program Π using the PIT tool and corresponding concretizations.
(v) Store line coverage and mutation score obtained from the PIT tool when executing T on Π for scenario s in a result file. Note that we store the averages of 10 runs for random selection.

Experimental Results

We conducted all experiments using an ordinary PC (Intel Core i7, 1.8 GHz, 8 GB Ram) running under Windows 10 and using Java 17 and SDK 19 for our implementation. Using the experimental process defined in Sect. "Evaluation Process", we obtain the following results. Table 5 shows the outcome for scenarios 1 to 4 using equal selection of values. In Table 6, we outline the results again for scenarios 1–4, using random value selection. Note that we have no results for Airline, Luhn, and Wronglock in Scenarios 3 and 4 in Table 6 because these programs became unresponsive with the given inputs. The same problem also happened in Scenario 4 of Table 5. In the following, we discuss the experimental results distinguishing the two value selection methods in detail.

We start discussing results obtained when relying on *equal selection* of values from domains (see Table 5). In all cases, except for Remainder, the line coverage and the mutation score increase or stay equivalent when going from a scenario with less to a finer granularity. In the case of Remainder, there is a decrease in the mutation score for Scenario 3 compared with Scenario 2, which is well visible in Fig. 1. The explanation might be that we miss executing some statements due to the internal structure and the selection of values. However, in all other cases, an increase in the domain indicates an increase in line coverage and mutation score when using combinatorial test suites. Hence, there is an observable influence of the input domain on the fault detection capabilities.

For *random selection*, the results depicted in Table 6 do not allow deriving a trend for increasing mutation score and line coverage when increasing the domain size for scenarios 1–4. There are programs like BMI or Remainder where an

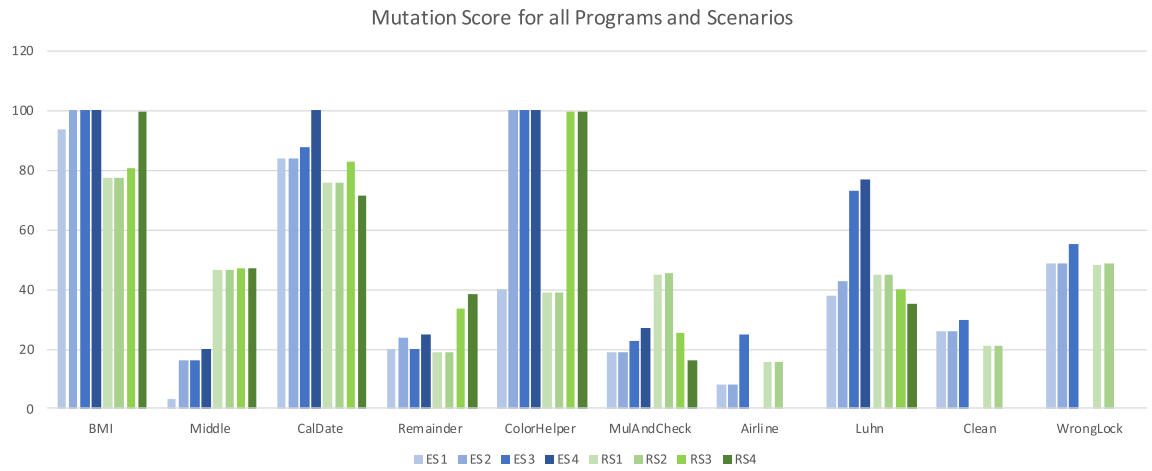


Fig. 1 Mutation scores for all scenarios and programs considering equal selection (ES*s*) and random selection (RS*s*), where *s* is the scenario number

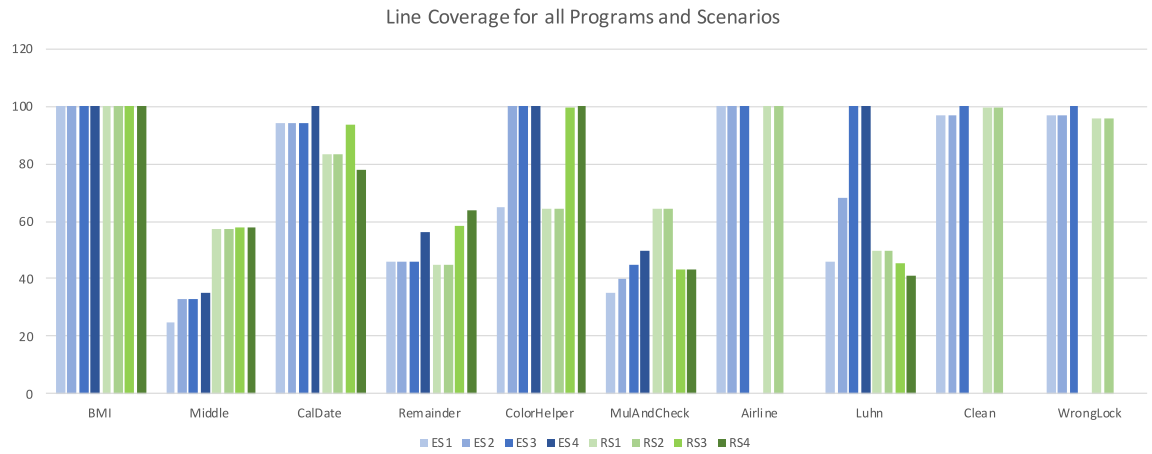


Fig. 2 Line coverage for all scenarios (1–4) and programs considering equal selection (ES) and random selection (RS)

increase in the domain size leads to an increase in the line coverage and mutation score. However, for many other programs, this is not the case. This is well visible in Fig. 1. What we can conclude again is the huge influence of the input model on the fault detection capabilities of a test suite. This is even more visible in the random selection case. Note that the line coverage for the different programs, scenarios, and value selection methods leads to the same conclusions (see Fig. 2).

When comparing the maximum value of line coverage and mutation score obtained using *random selection* and *equal selection*, we see that for the three programs, Middle, Remainder, and MulAndCheck random selection provides the best values. In all other cases, equal selection outperforms random selection. Both line coverage and mutation score could be substantially improved in the three cases. Hence, random selection of values might be a good choice, depending on the program's internal structure.

We observe notable differences in mutation score and line coverage between different selections. Increasing the domain size can also improve these measures. However, for *random selection*, this is not the case, especially when considering Luhn. This suggests that the input abstraction can dominate the input domain size in terms of influencing fault detection capability. In combination with the results of the audio plugin tests, these results indicate that some input selections lead to better fault detection capability. We later describe possible improvements that can allow us to find optimal input abstractions in Sect. "Discussion".

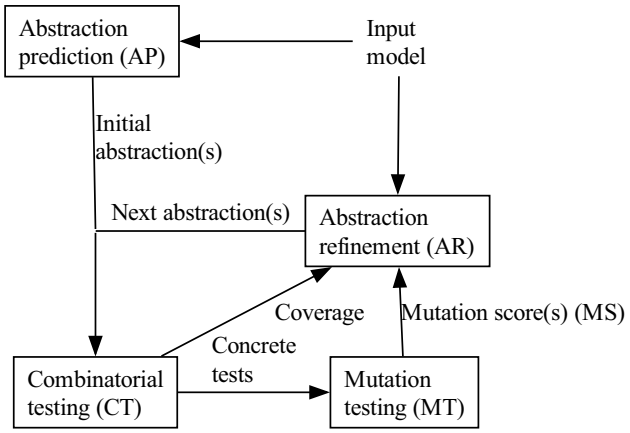


Fig. 3 A conceptual abstraction optimizer (AO)

Discussion

We now discuss possible improvements to fault detection capability of a CT test suite and the threats to the validity of our experimental evaluation in Sect. "Improvements" and "Threats to Validity", respectively.

Improvements

We established that the input abstraction, and in particular the size of the input domains, influences the fault detection capability of a CT test suite. We now investigate a general method that helps us to obtain an input abstraction optimized for fault detection given the SUT and its input model.

The audio plugin experiments in Sect. "Background" show that if relevant input values are known apriori, the input abstraction leads to a higher fault detection capability. Note that previous work exists as equivalence partitioning (EP). One empirical study [40] claims that EP is impractical compared to other methods like random testing. EP partitions the input domain and assumes every input value in a partition behaves the same under the testing goal. A typical EP approach requires a static analysis of the SUT's source code or preliminary testing to identify equivalences between input values. We investigate cases where the testing budget does not allow partition testing, and there is not enough time to thoroughly analyze the source code.

The audio plugin study [16] offers several ideas on an abstraction optimizer (AO). Note that these ideas are for continuous input domains only. We now describe and extend them to discrete input domains. Our aim is to provide a conceptual framework for automatic prediction and refinement of input abstractions while describing potential improvements on this framework in Sects. "Abstraction Prediction" and "Abstraction Refinement". We also elaborate on the details of these ideas, such as which reward and fitness functions are possible.

In Fig. 3, we show a conceptual AO. We divide the proposed improvements in this AO into two categories: abstraction prediction (AP) and abstraction refinement (AR).

Abstraction Prediction

Abstraction prediction (AP) takes an SUT's input model and generates a set of initial abstractions. The audio plugin experiments in Sect. "Background" and the new experiments in Sect. "Experiments and Evaluation" show that a naive input selection method such as random or equal leaves room for improvement. AP could predict important input values from other sources when this information is unavailable. These sources could be software requirements documents or simple static analysis methods. A limited dynamic analysis, i.e. executing the SUT a limited number of times, could also help. A recent study provides an algorithm to generate software specifications of a SUT by testing the SUT [41]. As long as the testing effort required to acquire such a specification is lower than the combinatorial testing effort, we can use this method to predict the most optimal AP.

Software requirements are often in natural language, so an NLP model can be used to process these requirements. Moreover, to extract input selections from NLP results, a database of known abstract input domains must be available, semantically labeled by an ontology. So, the task of the overall NLP approach is to identify semantically closest input domains from the requirements document. Once identified, AP may use the initial abstract input domain from the database instead of the input domain of the SUT's input model. AP, of course, must only consider abstractions that are subsets of the actual input domain. Note that more than one abstraction may fit the requirements, so in the later stages of AO may have to try these in parallel. Note that NLP techniques to extract test specifications is not new, and To the best of our knowledge, successfully used to enhance REST API testing [42].

For example, suppose one software requirement says that *the input a is the hypotenuse length of the right triangle in centimeters*. The NLP method could theoretically identify the word *length*. This word may match with a *float* and an *integer* in the ontology. If the ontology is detailed enough, AP may even choose an abstract domain where inputs are only positive numbers. An even more complex ontology should be able to identify that this is a *right triangle* and pick abstractions that aim to test the corner cases of such a construct.

There are challenges to using NLP to gather an initial abstraction for the input model. To the best of our knowledge, there are studies on ontology based test generation, and one study [43] suggests using ontologies for input modeling

in CT, but no such ontology is readily available and must be manually constructed [19]. Identifying abstractions for an independent input parameter should be trivial. However, the above example shows that input parameters often depend on each other, and the ontology may be required to identify constraints for multiple input parameters.

Abstraction Refinement

Once we have the initial abstractions, we apply CT to generate concrete tests. At this stage, we also execute the tests on the SUT. The experiments in Sect. "Experiments and Evaluation" show that some structural coverage measures may be available after CT. We also obtain which test covers which parts. Abstraction refinement (AR) will make use of this information. We apply MT on the concrete tests after CT and calculate the mutation scores. We also obtain which test kills which mutant.

First, some input values may contribute to neither coverage nor mutation score. AR can eliminate these values from the abstraction to make room for new input values. The previous work [16] denotes this approach as happy value exclusion (HVE), and does not consider coverage contributions.

AR uses the information from CT and MT as either a reward or a fitness value to refine the input abstractions. The previous work [16] suggests reinforcement learning (RL) or genetic algorithm (GA) to perform this refinement without going into the details. We now elaborate on this abstraction refinement process.

Reinforcement Learning Depending on the input domain, we define distinct action sets for RL. On an integer, intuitively, we can have three actions: incrementing, decrementing, or staying put. If the number is fractional, we can also use averaging between two fitting values. We can consider the string length as an integer when the input is a string. Then, we define actions that edit the character at the same position. We reward actions whose corresponding abstractions lead to an increase in MS or coverage.

Applying RL for AR can be challenging because the action space may become too large as the number of inputs increases. In that case, we may have to use a multi-objective RL solution, i.e., train multiple RLs in parallel for each input parameter. Also, if a string input's grammar is available, we could use RL on the input grammar instead of editing individual characters, which would shrink the action space.

Genetic Algorithm For GA, we must define how to mutate and crossover abstractions. We must also determine the fitness function. One challenge is to decide if a mutation replaces the old input value with an arbitrary one or uses the actions from the RL solution. This decision may require empirical analysis. The other challenge is to define crossovers. Aggregating two candidate numbers, e.g., taking the average, may linearize the input space.

The linearization may be harmful when the aggregate of two fault-detecting numbers is not a fault-detecting input value.

Instead, sampling the crossover number from a two-peaked distribution could be better. Mutating and crossing over strings are easier. We can use the rewards for RL as the fitness for GA.

Note that GA has to maintain a population of input abstractions while RL can maintain one input abstraction. We must re-execute the test suite for each refined input abstraction and calculate new scores. So, GA can be very costly in terms of test execution. However, RL may take longer to converge to an optimal solution. Hence, an empirical comparison of RL and GA for AR is necessary.

Overall, if the input is a complex structure, e.g., a class containing multiple variables, we can treat every member as a separate input parameter. However, capturing the dependencies between these parameters will be another challenge. Since AR involves executing some tests, we may now gather equivalence relations and prohibit AR from including equivalent input values in the same abstraction.

We terminate AR when the increase in MS at the current iteration becomes negligible. Stopping AR is essential due to the execution costs MT would add to a CI/CD pipeline. Once the test suite is optimized, it may be more practical to use it as is until the SUT changes significantly.

Threats to Validity

For our experiments, there might be different threats to internal and external validity, which we discuss in Sects. "Internal Validity" and "External Validity".

Internal Validity

We followed established scientific practices regarding internal validity. The main internal threats rely on the set of example programs and the used intervals from which we select the input domain. Most of the example programs are small. However, the selection of programs is not important to clarify that the CT input model substantially influences fault detection capabilities. The intervals used for selecting input values in our scenarios definitely impact the outcome, i.e., the measured line coverage and mutation score. However, since we analyze the different input models in relation, this influence holds for all scenarios and should at least not severely influence the findings.

In addition, the results obtained from the underlying experimental process rely on the tool PIT, which is often used in similar studies [44]. Although there is an influence on the outcome of tools used for obtaining coverage and mutation scores, it is very unlikely that there is

a tool-specific behavior for the programs used that would highly influence the final results.

External Validity

The selection of programs, intervals, and tools may limit the generalizability of some of our findings. This includes the observation that there is no clear picture regarding the influence of the domain size on fault detection capabilities in the case of random selection. Moreover, when using different programs, the same influence might not be visible anymore when using equal selection. Hence, we need to extend the experimental evaluation, especially for identifying relations between input model parameters and fault detection capabilities. However, there is no threat to the general objective to identify that there is such an influence.

Conclusions

In this paper, we explained the impact of input abstraction and in particular the domain size and selection on the fault detection capability of CT test suites. We described some of the experiments in previous work (see [16]) and added new experiments, strengthening our argument. The previous experiments were solely on audio plugin implementations, whereas the new experiments used software from various applications. We also measured line coverage (LC), showing that the test suites cover most parts of the SUT. We argued that even between naive input selection strategies like random and equal selection, there are notable differences in the fault detection capabilities. We elaborated on two possible improvements for obtaining an optimal input selection strategy: abstraction prediction and refinement. These results indicate that optimal input abstraction is an open problem with room for improvement.

In future work, we want to extend the experimental evaluation considering larger examples and different input types. In addition, we want to implement abstraction refinement to be combined with CT based on ontologies. The latter comes up with test cases where we need a good refinement to increase fault detection capabilities.

Acknowledgements The financial support by the Austrian Federal Ministry for Digital and Economic Affairs, the National Foundation for Research, Technology and Development, and the Christian Doppler Research Association is gratefully acknowledged.

Author contributions All authors contributed equally to this paper. They were involved in developing an initial manuscript, proofreading, and correcting. Carmen Baumann conducted the experimental evaluation, supervised by Franz Wotawa, who provided feedback. Franz Wotawa was also responsible for obtaining the QAMCAS funding.

Funding Open access funding provided by Graz University of Technology. The work has been carried out by the Christian Doppler Laboratory for Quality Assurance Methodologies for Autonomous Cyber-Physical Systems (QAMCAS). QAMCAS obtained funding has been obtained from the Austrian Federal Ministry for Digital and Economic Affairs, the National Foundation for Research, Technology and Development, and the Christian Doppler Research Association.

Data availability The data is available from the authors.

Declarations

Competing Interests On behalf of all authors, the corresponding author states that there is no conflict of interest

Research Involving Human and /or Animals Not applicable.

Informed Consent Not applicable.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by/4.0/.

References

1. Utting M, Legeard B. Practical model-based testing: a tools approach. Burlington: Morgan Kaufmann; 2010.
2. Schieferdecker I. Model-based testing. IEEE Softw. 2012;29:14–8.
3. Nie C, Leung H. A survey of combinatorial testing. ACM Comput Surv (CSUR). 2011;43:1–29.
4. Kuhn D, Kacker R, Lei Y. Introduction to combinatorial testing Chapman & Hall/CRC innovations in software engineering and software development series. Milton Park: Taylor & Francis; 2013.
5. Ali A, Maghawry HA, Badr N. Model-based test case generation approach for mobile applications load testing using ocl enhanced activity diagrams. In: 2021 tenth international conference on intelligent computing and information systems (ICICIS). 2021. p. 493–99.
6. Vilkomir S, Amstutz B. Using combinatorial approaches for testing mobile applications. In: 2014 IEEE seventh international conference on software testing, verification and validation workshops. 2014. p. 493–99.
7. Schieferdecker I. Model-based fuzz testing. In: Proceedings of the 2012 IEEE fifth international conference on software testing, verification and validation (ICST). 2012. p. 814.
8. Schieferdecker I, Grossmann J, Schneider M. Model-based security testing. In: Proceedings of the model-based testing workshop at ETAPS 2012. EPTCS. 2012. p. 1–12.
9. Bozic J, Garn B, Simos DE, Wotawa F. Evaluation of the ipo-family algorithms for test case generation in web security testing, 2015 IEEE Eighth International Conference on Software Testing, Verification and Validation Workshops (ICSTW), 1–10 (2015).

10. Liu G, Liu Q, Zhang W. Model-based testing and validation on artificial intelligence systems. In: Second international multi-symposiums on computer and computational sciences (IMSCCS 2007). 2007. p. 445–49.
11. Chandrasekaran J, Feng H, Lei Y, Kuhn DR, Kacker R. Applying combinatorial testing to data mining algorithms. In: 2017 IEEE international conference on software testing, verification and validation workshops (ICSTW). 2017. p. 253–61.
12. Bringmann E, Krämer A. Model-based testing of automotive systems. In: 2008 1st international conference on software testing, verification, and validation. 2008. p. 485–93.
13. Tao J, Li Y, Wotawa F, Felbinger H, Nica M. On the industrial application of combinatorial testing for autonomous driving functions. In: Proceedings of the international workshop on combinatorial testing (IWCT). IEEE; 2019.
14. Borazjany MN, Yu L, Lei Y, Kacker R, Kuhn R. Combinatorial testing of acts: a case study. In: 2012 IEEE fifth international conference on software testing, verification and validation. IEEE; 2012. p. 591–600.
15. Segall I, Tzoref-Brill R, Zlotnick A. Common patterns in combinatorial models. In: 2012 IEEE fifth international conference on software testing, verification and validation. 2012. p. 624–29.
16. Koroglu Y, Wotawa F. Continuous domain input abstraction and fault detection capability in combinatorial testing. In: 2023 38th IEEE/ACM international conference on automated software engineering workshops (ASEW). 2023. p. 167–70.
17. Grindal M., Offutt J. Input parameter modeling for combination strategies. In: Proceedings of the 25th conference on IASTED international multi-conference: software engineering. ACTA Press; 2007. p. 255–60.
18. Klück F, Li Y, Nica M, Tao J, Wotawa F. Using ontologies for test suites generation for automated and autonomous driving functions. In: Proceedings of the IEEE international symposium on software reliability engineering—industrial session. IEEE; 2018.
19. Wotawa F, Li Y. From ontologies to input models for combinatorial testing. In: 30th IFIP international conference on testing software and systems, ICTSS 2018; Conference date: 01-10-2018 Through 03-10-2018 (2018).
20. Ostrand TJ, Balcer MJ. The category-partition method for specifying and generating functional tests. Commun ACM. 1988;31:676–86.
21. Grochtmann M, Grimm K. Classification trees for partition testing. Softw Test Verif Reliab. 1993;3:63–82.
22. Klück F, Li Y, Tao J, Wotawa F. An empirical comparison of combinatorial testing and search-based testing in the context of automated and autonomous driving systems. Inf Softw Technol. 2023;160: 107225. https://doi.org/10.1016/j.infsof.2023.107225.
23. Lei Y, Kacker R, Kuhn DR, Okun V, Lawrence J. Ipog/ipog-d: efficient test generation for multi-way combinatorial testing. Softw Test Verif Reliab. 2008;18:125–48.
24. Yu L, Lei Y, Kacker RN, Kuhn DR. Acts: a combinatorial test generation tool. In: 2013 IEEE sixth international conference on software testing, verification and validation. IEEE; 2013. p. 370–75.
25. Tzoref-Brill R, Maoz S, Finkbeiner B, Pu G, Zhang L, editors. Lattice-based semantics for combinatorial model evolution. Automated technology for verification and analysis Cham: Springer International Publishing; 2015. p. 276–92.
26. Tzoref-Brill R, Maoz S. Syntactic and semantic differencing for combinatorial models of test designs. 2017. p. 621–31.
27. Budd T, DeMillo R, Lipton R, Sayward F. Theoretical and empirical studies on using program mutation to test the functional correctness of programs. In: Proc. seventh ACM symp. on princ. of prog. lang. (POPL). ACM; 1980.
28. Coles H, Laurent T, Henard C, Papadakis M, Ventresque A. Pit: a practical mutation testing tool for java. In: Proceedings of the 25th international symposium on software testing and analysis. ACM; 2016. p. 449–52.
29. Ma Y-S, Offutt J, Kwon YR. Mujava: an automated class mutation system. Softw Test Verif Reliab. 2005;15:97–133.
30. Groce A, Holmes J, Marinov D, Shi A, Zhang L. An extensible, regular-expression-based tool for multi-language mutant generation. In: 2018 IEEE/ACM 40th international conference on software engineering: companion (ICSE-Companion). IEEE; 2018. p. 25–8.
31. Offutt JA, Lee SD. An empirical evaluation of weak mutation. IEEE Trans Softw Eng. 1994;20:337–44.
32. Offutt AJ, Lee A, Rothermel G, Untch R, Zapf C. An experimental determination of sufficient mutation operators. ACM Trans Softw Eng Methodol. 1996;5:99–118.
33. Jia Y, Harman M. An analysis and survey of the development of mutation testing. IEEE Trans Softw Eng. 2011;37:649–78.
34. Hossain SB, Dwyer MB, Elbaum S, Nguyen-Tuong A. Measuring and mitigating gaps in structural testing. In: 2023 IEEE/ACM 45th international conference on software engineering (ICSE). 2023. p. 1712–23.
35. Fraser G, Arcuri A. Evosuite: automatic test suite generation for object-oriented software. In: ESEC/FSE '11. New York: Association for Computing Machinery; 2011. p. 416–19. https://doi.org/10.1145/2025113.2025179.
36. Sarfraz Khurshid GR, Darko Marinov. Software-artifact infrastructure repository. 2023. https://sir.csc.ncsu.edu/portal/index.php. Accessed 30 July 2023.
37. Min JL, Rajabi N, Rahmani A. Comprehensive study of sir: leading sut repository for software testing. J Phys Conf Ser. 2021;1869: 012072. https://doi.org/10.1088/1742-6596/1869/1/012072.
38. Yoo S, Harman M. Regression testing minimization, selection and prioritization: a survey. Softw Test Verif Reliab. 2012;22:67–120.
39. Lei Y, Kacker R, Kuhn DR, Okun V, Lawrence J. Ipog: a general strategy for t-way software testing. In: 14th annual IEEE international conference and workshops on the engineering of computer-based systems (ECBS'07). IEEE; 2007. p. 549–56.
40. Reid S. An empirical analysis of equivalence partitioning, boundary value analysis and random testing. 1997. p. 64–73.
41. Yandrapally R, Sinha S, Tzoref-Brill R, Mesbah A. Carving ui tests to generate api tests and api specification. 2023. p. 1971–82.
42. Kim M, et al. Enhancing rest api testing with nlp techniques. In: ISSTA 2023. New York: Association for Computing Machinery; 2023. p. 1232–43. https://doi.org/10.1145/3597926.3598131.
43. Li Y, Tao J, Wotawa F. Ontology-based test generation for automated and autonomous driving functions. Inf Softw Technol. 2020;117: 106200.
44. Laurent T, et al. Assessing and improving the mutation testing practice of pit. 2017. p. 430–35.

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.